

LAPORAN PRAKTIKUM STRUKTUR DATA DAN ALGORITMA
DASAR PEMOGRAMAN JAVASCRIPT



DI SUSUN OLEH :

NAMA	: FARA MUTIA
NIM	: 2025573010034
PRODI	: TEKNIK INFORMATIKA
DOSEN PENGAJAR	: M. REZA ZULMAN, S.ST.,M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
LHOKSEUMAWE

2026

LEMBARAN PENGESAHAN

MATA KULIAH : Praktikum Struktur Data dan Algoritma
JUDUL PRAKTIKUM : Laporan Praktikum Dasar Pemograman JavaScript
HARI PRAKTIKUM : Kamis
TANGGAL : 15 April 2026
PENYERAHAN
NAMA : Fara Mutia
NIM : 2025573010034
PRODI/KELAS : Teknik Informatika/I D
NILAI :

Mahasiswa

Bukit Rata, 10 April 2026
Dosen Pembimbing

Fara Mutia
NIM. 2025573010034

M. REZA ZULMAN S.ST.,M.Sc
NIP. 19920501 2022031005

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari tugas dalam mata kuliah Praktikum Struktur Data dan Algoritma yang bertujuan untuk memberikan pemahaman tentang sintax JavaScript mulai dari mengetahui variable, tipe data, operator, kondisional sampai perulangan.

Saya menyadari bahwa laporan ini tidak akan terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing, teman-teman, serta pihak-pihak yang telah memberikan bantuan baik secara langsung maupun tidak langsung dalam penyelesaian laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi semua pihak yang membacanya. Kritik dan saran yang konstruktif sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

Bukit Rata, 10 April 2026
Penulis

Fara Mutia
NIM. 2025573010034

DAFTAR ISI

LEMBARAN PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
BAB II LANDASAN TEORI	2
2.1 Pemograman JavaScript	2
2.1.1 <i>Function</i> sebagai <i>First-Class Citizens</i>	2
2.1.2 Dinamika Arrow Function dan Konteks Leksikal	2
2.1.3 Mekanisme Callback dan Pemrograman Asinkronus	3
2.1.4 Array	3
BAB III METOLOGI PERCOBAAN	4
3.1 Mempersiapkan Project Praktikum	4
3.2 Mengerjakan Contoh, Latihan dan Tugas	4
3.3 Sumbit ke GitHub	13
BAB IV PEMBAHASAN DAN ANALISIS	15
4.1 Analisis <i>Function</i>	15
4.2 Analisis Arrow Function	15
4.3 Analisis Array	15
4.4 Analisis Teoretis dan Praktis Algoritma Rekursif	16
4.5 Analisis Tugas Mandiri	16
BAB V PENUTUP.....	17
3.1 Kesimpulan	17
BAB VI DAFTAR PUSAKA.....	18

BAB I PENDAHULUAN

1.1 Latar Belakang

Laporan praktikum ini disusun untuk mendalami fondasi logika pemrograman pada modul struktur data dan algoritma, khususnya mengenai implementasi fungsi dan array. Dalam rekayasa perangkat lunak, efisiensi aplikasi sangat bergantung pada struktur kode yang modular, mudah dipelihara, dan hemat sumber daya. Selain aspek teknis sintaksis, laporan ini mengeksplorasi penerapan prinsip Don't Repeat Yourself (DRY) melalui fungsi dan penyimpanan koleksi informasi yang efisien dalam array.

Laporan ini akan mengulas secara naratif dan teknis seluruh rangkaian aktivitas praktikum, mulai dari inisialisasi proyek hingga analisis tugas mandiri yang melibatkan algoritma rekursif yang kompleks. Pembahasan tidak hanya terbatas pada sintaksis kode, tetapi juga mencakup wawasan tingkat kedua dan ketiga mengenai implikasi desain perangkat lunak, manajemen tumpukan eksekusi (*execution stack*), serta perbandingan efisiensi antara metode iterasi modern dan konvensional. Melalui dokumentasi yang sistematis, diharapkan pembaca dapat menangkap esensi dari setiap modul percobaan serta memahami landasan teoretis yang mendasarinya.

1.2 Tujuan Praktikum

1. Mampu mendefinisikan fungsi dengan parameter yang tepat serta nilai kembalian (return value) yang konsisten.
2. Penggunaan arrow function untuk efisiensi penulisan kode dan pemahaman manajemen konteks this.
3. Pengenalan konsep callback sebagai dasar arsitektur event-driven pada Node.js.
4. Penguasaan metode iterasi tingkat tinggi (high-order functions) seperti forEach, map, filter, dan reduce untuk transformasi data.
5. Mampu menerapkan teknik rekursi dengan pendekatan divide and conquer untuk menangani struktur data kompleks.

BAB II LANDASAN TEORI

2.1 Pemograman JavaScript

2.1.1 *Function* sebagai *First-Class Citizens*

Dalam JavaScript, fungsi memiliki kedudukan yang sangat istimewa. Fungsi dianggap sebagai first-class citizens, yang berarti fungsi dapat diperlakukan selayaknya data biasa: disimpan dalam variabel, dilewatkan sebagai argumen ke fungsi lain, atau bahkan dikembalikan dari fungsi lain sebagai nilai. Karakteristik ini memungkinkan terciptanya abstraksi tingkat tinggi dan modularitas yang luar biasa dalam desain perangkat lunak. Secara teknis, sebuah fungsi didefinisikan dengan kata kunci `function`, diikuti oleh nama unik, parameter dalam kurung, dan badan fungsi dalam kurung kurawal.

Penting untuk dicatat bahwa penggunaan parameter memberikan fleksibilitas pada fungsi untuk memproses berbagai masukan tanpa mengubah logika internalnya. JavaScript juga menyediakan fitur default parameter, yang secara otomatis memberikan nilai cadangan jika argumen tidak diberikan oleh pemanggil. Hal ini sangat berguna untuk membangun API yang tangguh dan mencegah terjadinya galat akibat nilai `undefined`. Dari perspektif aliran kontrol, kata kunci `return` berfungsi sebagai penghenti eksekusi sekaligus pengirim hasil ke konteks pemanggil; segala instruksi yang berada di bawah `return` di dalam blok yang sama tidak akan pernah dijalankan oleh mesin JavaScript.

2.1.2 *Dinamika Arrow Function dan Konteks Leksikal*

Arrow function menawarkan alternatif penulisan fungsi yang lebih ringkas menggunakan sintaksis `=>`. Meskipun sering dianggap hanya sebagai penyederhanaan, *arrow function* memiliki perbedaan fundamental dalam hal pengikatan konteks `this`. Tidak seperti fungsi tradisional yang memiliki konteks `this` dinamis bergantung pada bagaimana fungsi tersebut dipanggil, *arrow function* mewarisi konteks `this` dari lingkup tempat ia didefinisikan (lingkup leksikal).

Selain itu, terdapat perbedaan teknis dalam hal perilaku saat kode dijalankan. Fungsi deklarasi tradisional mengalami hoisting penuh sehingga bisa dipanggil sebelum didefinisikan, sedangkan *arrow function* berperilaku seperti variabel yang tidak mengalami *hoisting*. Dari sisi kegunaan, fungsi tradisional dapat digunakan sebagai constructor dengan kata kunci `new` dan memiliki akses ke objek `arguments`, fitur yang tidak didukung oleh *arrow function*. Perbedaan-perbedaan ini menunjukkan bahwa

meskipun *arrow function* lebih modern dan ringkas, keduanya memiliki peran spesifik yang disesuaikan dengan kebutuhan arsitektur kode.

Penggunaan *arrow function* sangat disarankan dalam skenario di mana keringkasn kode menjadi prioritas, seperti pada metode iterasi array atau fungsi anonim pendek. Namun, untuk metode objek yang membutuhkan akses ke instansi objek itu sendiri melalui *this*, fungsi tradisional tetap menjadi pilihan utama.

2.1.3 Mekanisme Callback dan Pemrograman Asinkronus

Callback adalah fungsi yang dilewatkan sebagai argumen ke fungsi lain dengan harapan akan dieksekusi pada waktu tertentu setelah tugas utama selesai. Analogi yang sering digunakan adalah sistem pemesanan makanan di restoran: pelanggan memberikan nomor meja (callback), dan pelayan akan mengantarkan pesanan saat sudah siap (event selesai), sementara pelanggan bebas melakukan aktivitas lain.

Dalam praktiknya, callback dapat bersifat sinkronus maupun asinkronus. Contoh sinkronus adalah penggunaan callback pada metode **Array.prototype.map()**, di mana fungsi dijalankan seketika untuk setiap elemen. Sementara itu, fungsi **setTimeout()** adalah contoh klasik dari callback asinkronus, di mana eksekusi ditunda selama durasi tertentu tanpa menghentikan jalannya program utama. Ketidakpahaman terhadap urutan eksekusi asinkronus sering kali menyebabkan kebingungan bagi pengembang pemula, namun ini adalah kunci untuk membangun aplikasi yang responsif dan mampu menangani ribuan permintaan secara bersamaan.

2.1.4 Array

Array adalah objek untuk menyimpan daftar data secara berurutan. Berbeda dengan bahasa pemrograman tingkat rendah yang memerlukan alokasi memori statis, array di JavaScript bersifat dinamis dan dapat menyimpan berbagai tipe data sekaligus dalam satu kontainer. Setiap elemen dalam array diasosiasikan dengan indeks numerik yang dimulai dari 0.

Operasi dasar pada array melibatkan manipulasi elemen di berbagai posisi. Metode **push()** dan **pop()** bekerja pada akhir array dengan kompleksitas waktu $O(1)$ karena tidak memerlukan pergeseran elemen lain. Sebaliknya, **unshift()** dan **shift()** yang bekerja pada awal array memiliki kompleksitas $O(n)$ karena mesin JavaScript harus memperbarui indeks seluruh elemen yang tersisa setelah operasi dilakukan.

BAB III METOLOGI PERCOBAAN

3.1 Mempersiapkan Project Praktikum

1. Buka App Git Bash. Kemudian navigasikan ke root folder repository.

```
newuser@DESKTOP-JP3BR80 MINGW64 ~  
$ cd '/e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma'
```

2. Buat folder baru dengan nama praktikum-2 untuk menyimpan semua file pada pertemuan ini dan masuk kedalamnya.

```
newuser@DESKTOP-JP3BR80 MINGW64 ~  
$ cd '/e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma'  
  
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma (main)  
$ mkdir praktikum-3  
  
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma (main)  
$ cd praktikum-3
```

3. Inisialisasi npm project.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma/praktikum-3 (main)  
$ npm init -y  
Wrote to E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgoritma\praktikum-3\package.json:  
  
{  
  "name": "praktikum-3",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}
```

3.2 Mengerjakan Contoh, Latihan dan Tugas

3.2.1 Mendefinisikan Function

1. Membuat file baru dengan nama 01-function-dasar.js di dalam folder praktikum-3

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma/praktikum-3 (main)  
$ touch 01-function-dasar.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya dan kerjakan latihannya.


```

1 //1. Function tanpa parameter dan return value
2 function sapa(){
3     console.log('Halo! Selamat datang di praktikum JavaScript.');
```

4 }
5 sapa();
6 sapa();
7
8 //2. Function tanpa parameter dan return value
9 function sapaNama(nama){
10 console.log(`Halo, \${nama}! Selamat belajar`);
11 }
12 sapaNama('Budi');
13 sapaNama('Siti');
14
15 function tambah(a, b){
16 const hasil = a + b;
17 return hasil;
18 }
19
20 const hasilPenjumlahan = tambah(10, 25);
21 console.log('10 + 25 =', hasilPenjumlahan);
22 console.log('7 + 13 =', tambah(7,13));
23
24 function hitung(nilai, pengali = 2){
25 return nilai * pengali;
26 }
27 console.log(hitung(5));
28 console.log(hitung(5,3));
29
30 const pesanGlobal = 'Saya ada di mana saja';
31

```

32 function cekScope(){
33     const pesanLokal = 'Saya hanya ada dalam function ini';
34     console.log(pesanGlobal);
35     console.log(pesanLokal);
36 }
37 cekScope();
38 console.log(pesanGlobal);
39
40 //Latihan
41 function kurang(a, b) {
42     return a - b;
43 }
44
45 function kali(a, b) {
46     return a * b;
47 }
48
49
50 function bagi(a, b) {
51     if (b === 0) {
52         console.log('Error: tidak bisa membagi dengan nol');
53         return null;
54     }
55     return a / b;
56 }

```

```

58 function kalkulator(a, operator, b) {
59     switch (operator) {
60         case '+':
61             return tambah(a, b);
62         case '-':
63             return kurang(a, b);
64         case '*':
65             return kali(a, b);
66         case '/':
67             return bagi(a, b);
68         default:
69             return "Operator tidak valid";
70     }
71 }
72
73
74 function tambah(a, b) {
75     return a + b;
76 }
77 console.log("=== LATIHAN ===");
78 console.log("Hasil 10 + 5 :", kalkulator(10, '+', 5));
79 console.log("Hasil 10 - 5 :", kalkulator(10, '-', 5));
80 console.log("Hasil 10 * 5 :", kalkulator(10, '*', 5));
81 console.log("Hasil 10 / 5 :", kalkulator(10, '/', 5));
82 console.log("Hasil 10 / 0 :", kalkulator(10, '/', 0));

```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 01-variabel.js** di terminal.

```

node .\01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar
Halo, Siti! Selamat belajar
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada dalam function ini
Saya ada di mana saja
=== LATIHAN ===
Hasil 10 + 5 : 15
Hasil 10 - 5 : 5
Hasil 10 * 5 : 50
Hasil 10 / 5 : 2
Error: tidak bisa membagi dengan nol
Hasil 10 / 0 : null

```

3.2.2 Arrow Function & Callback

1. Membuat file baru dengan nama 02-arrow-callback.js di dalam folder praktikum-3

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-3 (main)
$ touch 02-arrow-callback.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya dan kerjakan latihannya.

```
1  function kuadratBiasa(x){
2      return x * x;
3  }
4  const kuadrat = (x) =>{
5      return x * x;
6  };
7
8  const kuadratRingkas = x => x * x;
9
10 console.log('=== Perbandingan Penulisan ===');
11 console.log('Biasa   :', kuadratBiasa(5));
12 console.log('Arrow   :', kuadrat(6));
13 console.log('Ringkas  :', kuadratRingkas(7));
14
15 const Luas = (panjang, lebar) => panjang * lebar;
16 const salam =(nama, waktu) => `Selamat ${waktu}, ${nama}!`;
17
18 console.log('Luas 4 x 6 :', Luas(4,6));
19 console.log(salam('Budid', 'Pagi'));
20
21 function lakukanOperasi (angka, operasiCallback){
22     console.log(`Angka awal : ${angka}`);
23     const hasil = operasiCallback(angka);
24     console.log(`Hasil setelah operasi : ${hasil}`);
25 }
26
27 console.log('\n=== Callback===');
28 lakukanOperasi(7, kuadratRingkas);
29 lakukanOperasi(10, x => x + 100);
30 lakukanOperasi(20, function(x){
31     return x /2;
32 })
33
34 console.log('\n===setTimeout (Callback)===');
35 console.log('Pesan 1: Sebelum timer');
36
37 setTimeout(() => {
38     console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)');
39 }, 1000);
40 console.log('Pesan 2: Setelah mendaftarkan timer');
41
42 //latihan
43 const keHurufBesar = (str) => str.toUpperCase();
44
45 const tambahSeru = (str) => str + "!!!";
46
47 const hitungKata = (str) => str.split(' ').length;
48
49 function prosesKalimat(kalimat, transformasiCallback) {
50     const hasil = transformasiCallback(kalimat);
51     console.log(hasil);
52 }
53
54 const kalimatUji = 'belajar javascript itu menyenangkan hati';
55
56 console.log("\nHasil keHurufBesar:");
57 prosesKalimat(kalimatUji, keHurufBesar);
58
59 console.log("\nHasil tambahSeru:");
60 prosesKalimat(kalimatUji, tambahSeru);
61
62 console.log("\nHasil hitungKata:");
63 prosesKalimat(kalimatUji, hitungKata);
```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 02-arrow-callback.js** di terminal.

```
node .\02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 36
Ringkas : 49
Luas 4 x 6 : 24
Selamat Pagi, Budid!

=== Callback===
Angka awal : 7
Hasil setelah operasi : 49
Angka awal : 10
Hasil setelah operasi : 110
Angka awal : 20
Hasil setelah operasi : 10

===setTimeout (Callback)===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer

Hasil keHurufBesar:
BELAJAR JAVASCRIPT ITU MENYENANGKAN HATI

Hasil tambahSeru:
belajar javascript itu menyenangkan hati!!!

Hasil hitungKata:
5
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
```

3.2.3 Array Dasar

1. Membuat file baru dengan nama 03-array-dasar.js di dalam folder praktikum-3.

```
newuser@DESKTOP-3P3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-3 (main)
$ touch 03-array-dasar.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya dan kerjakan latihannya.

```
const mahasiswa = ['Budi', 'Siti', 'Ahmad', 'Rina'];
const nilai = [85, 92, 78, 95, 88];

console.log('===Array Awal===');
console.log('Mahasiswa :', mahasiswa);
console.log('Nilai :', nilai);
console.log('Jumlah mahasiswa :', mahasiswa.length);

console.log('\n=== Akses Elemen ===');
console.log('Elemen pertama :', mahasiswa[0]);
console.log('Elemen ketiga :', mahasiswa[2]);
console.log('Elemen terakhir:', mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = 'Siti Rahayu';
console.log('\nSetelah diubah :', mahasiswa);

console.log('\n=== Manipulasi Array ===');
mahasiswa.push('Doni');
console.log('Setelah unshift :', mahasiswa);
```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 03-array-dasar.js** di terminal.

```
===Array Awal===
Mahasiswa : [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa : 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Budi
Elemen terakhir: Rina

Setelah diubah : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah unshift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (pop) : Budi
Setelah shift : [ 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 1
Ada Rina? : true
Ada budi? : false

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4 : [ 92, 78, 95 ]
```

```
=== Latihan ===

=== Daftar Belanja Awal ===
1. Beras
2. Gula
3. Minyak
4. Telur
5. Kopi

Setelah tambah item:
1. Beras
2. Gula
3. Minyak
4. Telur
5. Kopi
6. Cabai
7. Ice cream

Item yang dihapus: Beras
Apakah "Susu" ada di daftar? false
Jumlah item akhir: 6
```

3.2.4 Method Iterasi Array

1. Membuat file baru dengan nama 04-array-method.js di dalam folder praktikum-3.

```
newuser@DESKTOP-JP3BRB0 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadana/algorithm/praktikum-3 (main)
$ touch 04-array-method.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya dan kerjakan latihan.

```
1  const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
2  console.log('===forEach: Tampilkan Semua Nilai ===');
3  nilaiMahasiswa.forEach((nilai, indeks) => {
4    console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
5  });
6  const gradeHuruf = nilaiMahasiswa.map(nilai => {
7    if(nilai >= 90) return 'A';
8    if(nilai >= 80) return 'B';
9    if(nilai >= 70) return 'C';
10   if(nilai >=60) return 'D';
11   return 'E';
12 });
13 console.log('\n=== map: Nilai ke Grade===');
14 console.log('Nilai :', nilaiMahasiswa);
15 console.log('Grade :', gradeHuruf);
16 const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
17 const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);
18 console.log('\n===Filter Lulus dan Tidak Lulus===');
19 console.log('Semua nilai :', nilaiMahasiswa);
20 console.log('Nilai lulus :', nilaiLulus);
21 console.log('Nilai gagal :', nilaiGagal);
22 const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
23   return akumulator + nilai;
24 }, 0);
25 const rataRata = totalNilai / nilaiMahasiswa.length;
26 console.log('\nTotal nilai :', totalNilai);
27 console.log('Rata-rata :', rataRata.toFixed());
28 const rataRataNilaiLulus = nilaiMahasiswa
29   .filter(nilai => nilai >= 60)
30   .reduce((acc, val, idx, arr) => {
31     return acc + val / arr.length;
32   }, 0);
33 console.log('\n===Method Chaining===');
34 console.log('Rata-rata nilai lulus:', rataRataNilaiLulus.toFixed(2));
35
36 //Latihan
37 console.log('=== Latihan ===');
38 const produk = [
39   { nama: 'Laptop', harga: 8500000, stok: 5 },
40   { nama: 'Mouse', harga: 150000, stok: 0 },
41   { nama: 'Keyboard', harga: 450000, stok: 12 },
42   { nama: 'Monitor', harga: 3200000, stok: 0 },
43   { nama: 'Headset', harga: 350000, stok: 8 }
44 ];
45 const produkTersedia = produk.filter(item => item.stok > 0);
46 console.log("Produk tersedia:");
47 console.log(produkTersedia);
48 const namaProdukTersedia = produkTersedia.map(item => item.nama);
49 console.log("\nNama produk yang tersedia:");
50 console.log(namaProdukTersedia);
51 const totalInventaris = produkTersedia.reduce((total, item) => total + (item.harga * item.stok), 0);
52 console.log("\nTotal nilai inventaris produk tersedia:");
53 console.log(`Rp${totalInventaris.toLocaleString('id-ID')}`);
54 console.log("\nDaftar semua produk:");
55 produk.forEach(item => {
56   const status = item.stok > 0 ? "TERSEDIA" : "HABIS";
57   const hargaFormatted = `Rp${item.harga.toLocaleString('id-ID')}`;
58   console.log(`${status} ${item.nama} - ${hargaFormatted} (${item.stok} unit)`);
59 });
60
```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 04-array-method.js** di terminal.

```

===forEach: Tampilkan Semua Nilai ===
Mahasiswa-1: 75
Mahasiswa-2: 90
Mahasiswa-3: 55
Mahasiswa-4: 82
Mahasiswa-5: 68
Mahasiswa-6: 95
Mahasiswa-7: 48
Mahasiswa-8: 77

=== map: Nilai ke Grade===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

===Filter Lulus dan Tidak Lulus===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]

Total nilai : 590
Rata-rata : [Function: toFixed]

===Method Chaining===
Rata-rata nilai lulus: 81.17

=== Latihan ===
Produk tersedia:
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Nama produk yang tersedia:
[ 'Laptop', 'Keyboard', 'Headset' ]

Total nilai inventaris produk tersedia:
Rp50.700.000

Daftar semua produk:
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000 (0 unit)
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000 (0 unit)
[TERSEDIA] Headset - Rp350.000 (8 unit)

```

3.2.5 Rekursi

1. Membuat file baru dengan nama 05-rekursi.js di dalam folder praktikum-3.

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-3 (main)
$ touch 05-rekursi.js

```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.
3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 05-rekursi.js** di terminal.

3.2.6 Tugas Praktikum

1. Buat folder tugas/

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-2 (main)
$ mkdir tugas

```

2. Lalu tambahkan dua file tugas di dalam folder tugas/ tersebut yaitu tugas-1.js dan tugas-2.js.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-2/tugas (main)
$ touch tugas-1.js

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-2/tugas (main)
$ touch tugas-2.js
```

3. Program tugas 1 beserta hasil outputnya.

```
1  const dataMahasiswa = [
2    { nama: "Anisa", nilai: 85 },
3    { nama: "Budi", nilai: 70 },
4    { nama: "Clara", nilai: 90 },
5    { nama: "Dewi", nilai: 60 },
6    { nama: "Eka", nilai: 75 },
7    { nama: "Fara", nilai: 95 }
8  ];
9  function hitungStatistik(arrMahasiswa) {
10    const total = arrMahasiswa.length;
11
12    const hasil = arrMahasiswa.reduce(
13      (acc, curr) => {
14        acc.totalNilai += curr.nilai;
15        if (curr.nilai > acc.tertinggi) acc.tertinggi = curr.nilai;
16        if (curr.nilai < acc.terendah) acc.terendah = curr.nilai;
17        return acc;
18      },
19      {
20        totalNilai: 0,
21        tertinggi: -Infinity,
22        terendah: Infinity
23      }
24    );
25    return {
26      total,
27      rataRata: hasil.totalNilai / total,
28      tertinggi: hasil.tertinggi,
29      terendah: hasil.terendah
30    };
31  }
32  function filterLulus(arrMahasiswa, batasLulus) {
33    return arrMahasiswa.filter(mhs => mhs.nilai >= batasLulus);
34  }
35
36  function tambahGrade(arrMahasiswa) {
37    return arrMahasiswa.map(mhs => {
38      let grade = "";
39
40      if (mhs.nilai >= 85) grade = "A";
41      else if (mhs.nilai >= 75) grade = "B";
42      else if (mhs.nilai >= 65) grade = "C";
43      else if (mhs.nilai >= 50) grade = "D";
44      else grade = "E";
45
46      return {
47        ...mhs,
48        grade
49      };
50    });
51  }
52
53  const statistik = hitungStatistik(dataMahasiswa);
54  const lulus = filterLulus(dataMahasiswa, 75);
55  const denganGrade = tambahGrade(dataMahasiswa);
56
57  console.log("=== Statistik ===");
58  console.log(`Total Mahasiswa : ${statistik.total}`);
59  console.log(`Rata-rata Nilai : ${statistik.rataRata.toFixed(2)}`);
60  console.log(`Nilai Tertinggi : ${statistik.tertinggi}`);
61  console.log(`Nilai Terendah : ${statistik.terendah}`);
62
63  console.log("\n=== Mahasiswa Lulus ===");
64  lulus.forEach(mhs => {
65    console.log(`Nama: ${mhs.nama}, Nilai: ${mhs.nilai}`);
66  });
67
68  console.log("\n=== Data dengan Grade ===");
69  denganGrade.forEach(mhs => {
70    console.log(`Nama: ${mhs.nama}, Nilai: ${mhs.nilai}, Grade: ${mhs.grade}`);
71  });
```

```
=== Statistik ===
Total Mahasiswa : 6
Rata-rata Nilai : 79.17
Nilai Tertinggi : 95
Nilai Terendah : 60

=== Mahasiswa Lulus ===
Nama: Anisa, Nilai: 85
Nama: Clara, Nilai: 90
Nama: Eka, Nilai: 75
Nama: Fara, Nilai: 95

=== Data dengan Grade ===
Nama: Anisa, Nilai: 85, Grade: A
Nama: Budi, Nilai: 70, Grade: C
Nama: Clara, Nilai: 90, Grade: A
Nama: Dewi, Nilai: 60, Grade: D
Nama: Eka, Nilai: 75, Grade: B
Nama: Fara, Nilai: 95, Grade: A
```


4. Program tugas 2 beserta hasil outputnya.

```
1 function pangkat(basis, eksponen) {
2   if (eksponen === 0) return 1; // base case
3   return basis * pangkat(basis, eksponen - 1);
4 }
5
6
7 function balikString(str) {
8   if (str.length <= 1) return str; // base case
9   return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
10 }
11
12
13 function cekPalindrom(str) {
14   const hasilBalik = balikString(str);
15   return str === hasilBalik;
16 }
17
18
19 console.log("=== Uji Pangkat ===");
20 console.log(`2^3 = ${pangkat(2, 3)}`);
21 console.log(`5^2 = ${pangkat(5, 2)}`);
22 console.log(`10^0 = ${pangkat(10, 0)}`);
23
24 console.log("\n=== Uji Balik String ===");
25 console.log(`"halo" -> "${balikString("halo")}"`);
26 console.log(`"javascript" -> "${balikString("javascript")}"`);
27
28 console.log("\n=== Uji Palindrom ===");
29 const kata = ["katak", "civic", "level", "halo", "dunia", "kasur rusak"];
30
31 kata.forEach(k => {
32   console.log(`${k} -> ${cekPalindrom(k) ? "Palindrom" : "Bukan Palindrom"}`);
33 });
34
```

```
=== Uji Pangkat ===
2^3 = 8
5^2 = 25
10^0 = 1

=== Uji Balik String ===
"halo" -> "olah"
"javascript" -> "tpircsavaJ"

=== Uji Palindrom ===
katak -> Palindrom
civic -> Palindrom
level -> Palindrom
halo -> Bukan Palindrom
dunia -> Bukan Palindrom
kasur rusak -> Palindrom
```

3.3 Sumbit ke GitHub

Setelah semua tugas, latihan, dan contoh selesai, lakukan push ke *repository* GitHub.

1. Ketik `cd ..` di Git Bush agar tidak berada dalam `/praktikum-3` tetapi di root folder repository. Karena Git diinisialisasi di folder utama (root). Kalau kita di dalam folder `praktikum-3`, Git mungkin tidak akan melihat perubahan di folder lain.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-2 (main)
$ cd ..

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma (main)
$ |
```

2. Ketik `git status` untuk cek status file, lalu ketik `git add praktikum -3/` untuk menambahkan

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    praktikum-3/

no changes added to commit (use "git add" and/or "git commit -a")

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma (main)
$ git add praktikum-3/
warning: in the working copy of 'praktikum-3/package.json', LF will be replaced
by CRLF the next time Git touches it
```

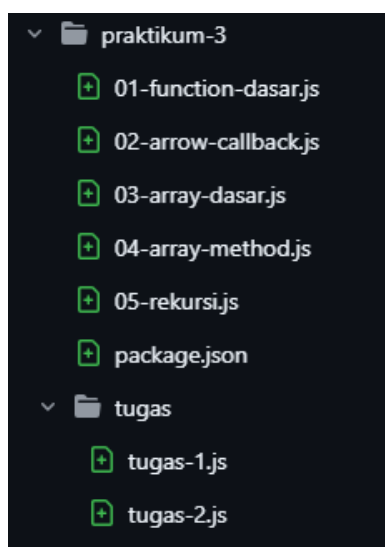
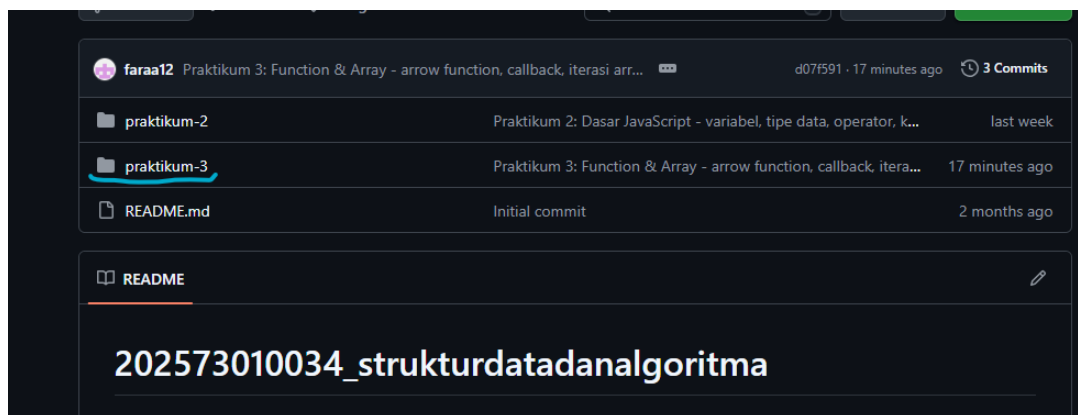
3. Buat commit dengan pesan yang deskriptif. Seperti *git commit -m "Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi"*.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma (main)
$ git commit -m "Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi"
[main d07f591] Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi
8 files changed, 431 insertions(+)
create mode 100644 praktikum-3/01-function-dasar.js
create mode 100644 praktikum-3/02-arrow-callback.js
create mode 100644 praktikum-3/03-array-dasar.js
create mode 100644 praktikum-3/04-array-method.js
create mode 100644 praktikum-3/05-rekursi.js
create mode 100644 praktikum-3/package.json
create mode 100644 praktikum-3/tugas/tugas-1.js
create mode 100644 praktikum-3/tugas/tugas-2.js
```

4. Lalu lakukan push ke repository GitHub.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.97 KiB | 1.24 MiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/faraa12/202573010034_strukturdatadanalgoritma.git
 a88c274..d07f591  main -> main
```

5. Hasil akhir



BAB IV PEMBAHASAN DAN ANALISIS

4.1 Analisis *Function*

Praktikum ini menegaskan peran fungsi sebagai blok pembangun modular yang mampu membungkus logika repetitif melalui prosedur sederhana maupun fungsi dengan parameter dinamis. Implementasi fungsi dengan nilai kembalian (return value) memungkinkan terjadinya komposisi fungsi, sementara default parameter memberikan fleksibilitas pada sistem dalam menangani input yang tidak lengkap.

Dalam aspek keamanan kode, eksperimen scope menunjukkan bahwa enkapsulasi variabel menggunakan ``const`` di dalam fungsi sangat krusial untuk mencegah polusi global dan `ReferenceError`. Konsep shadowing turut diuji untuk memberikan prioritas pada variabel lokal, sementara pada tingkat aplikasi, prinsip defensive programming diterapkan pada fungsi kalkulator untuk menangani kasus pembagi nol guna menghindari kerusakan data pada perhitungan yang lebih kompleks.

4.2 Analisis Arrow Function

Arrow function pada praktikum ini menyoroti modernisasi sintaksis JavaScript yang lebih ringkas, terutama untuk logika satu baris yang bersifat implisit. Namun, fokus utama bergeser pada simulasi event-driven menggunakan callback asinkronus melalui ``setTimeout``. Hasilnya membuktikan karakteristik non-blocking execution pada Node.js, di mana kode tetap berjalan tanpa menunggu proses timer selesai.

Penerapan Strategy Pattern juga terlihat dalam latihan "Pipeline Transformasi Data", di mana fungsi ``prosesKalimat`` bertindak sebagai orkestrator yang menerima berbagai fungsi transformasi sebagai callback. Pendekatan ini memungkinkan pengembangan sistem yang fleksibel dan dinamis tanpa harus mengubah logika inti pemrosesan data.

4.3 Analisis Array

Eksperimen ini membedah array sebagai struktur data terindeks yang dinamis namun menuntut ketelitian dalam validitas data. Ditemukan perbedaan performa yang signifikan antara metode manipulasi; ``push`` dan ``pop`` terbukti efisien untuk manajemen stack, sedangkan ``unshift`` dan ``shift`` lebih lambat karena adanya pergeseran memori pada indeks awal.

Selain itu, penggunaan metode seperti ``indexOf``, ``includes``, dan khususnya ``slice``, sangat ditekankan untuk menjaga integritas data melalui konsep immutable state. Dengan menggunakan ``slice``, operasi pada array dapat dilakukan pada salinan data tanpa merusak sumber aslinya, yang merupakan praktik terbaik dalam arsitektur perangkat lunak modern.

4.4 Analisis Teoretis dan Praktis Algoritma Rekursif

Eksperimen rekursi mengeksplorasi pemecahan masalah melalui mekanisme call stack, di mana setiap fungsi memanggil dirinya sendiri hingga mencapai base case. Berbagai algoritma diuji, mulai dari perhitungan faktorial, deret Fibonacci, hingga penelusuran array. Eksperimen ini memberikan pemahaman krusial bahwa rekursi yang tidak efisien (seperti pada Fibonacci naif) dapat menyebabkan redundansi komputasi eksponensial.

Selain itu, praktikum ini memperingatkan risiko teknis stack overflow (``RangeError``) jika kedalaman rekursi melampaui batas memori Node.js. Hal ini menegaskan bahwa meskipun rekursi sangat elegan untuk struktur data hierarkis, penggunaannya harus dibatasi atau dioptimalkan untuk iterasi linier yang sangat panjang.

4.5 Analisis Tugas Mandiri

Implementasi tugas mandiri menggabungkan seluruh konsep dalam skenario dunia nyata. Pada sistem statistik mahasiswa, penggunaan ``reduce`` secara optimal berhasil menghasilkan objek ringkasan (total, rata-rata, tertinggi, terendah) hanya dalam satu lintasan iterasi, yang jauh lebih efisien daripada penggunaan banyak filter.

Pada tugas kedua, logika rekursi diterapkan lebih jauh untuk menangani operasi matematika pangkat dan manipulasi string pada pengecekan palindrom. Penggunaan teknik pemotongan string (``slice``) dalam fungsi rekursif ``balikString`` menunjukkan bagaimana masalah kompleks dapat dipecah menjadi bagian-bagian kecil yang dikerjakan secara berulang hingga mencapai solusi akhir.

BAB V PENUTUP

3.1 Kesimpulan

Praktikum Modul 3 mengenai fungsi dan array telah memberikan landasan yang sangat kuat bagi pemahaman struktur data dan algoritma dalam ekosistem JavaScript. Fungsi telah terbukti sebagai elemen modular paling vital yang memungkinkan pemisahan tanggung jawab (separation of concerns) dan penggunaan ulang kode. Pengenalan terhadap sintaksis modern dan pola asinkronus melalui callback telah membuka wawasan mengenai cara kerja Node.js dalam menangani operasi berbasis waktu dan event.

Eksplorasi mendalam terhadap array menunjukkan bahwa struktur data ini bukan sekadar daftar statis, melainkan objek dinamis dengan ekosistem metode yang sangat kaya untuk manipulasi data yang bersih dan deklaratif. Meskipun metode modern seperti `reduce` dan `map` membawa sedikit beban performa, keuntungan dalam hal keterbacaan dan pemeliharaan kode menjadikannya pilihan utama dalam pengembangan perangkat lunak saat ini.

Terakhir, penguasaan teknik rekursi melengkapi perangkat logika pengembang, memungkinkan penyelesaian masalah kompleks dengan cara yang elegan dan intuitif. Namun, kesadaran akan batasan tumpukan eksekusi dan efisiensi memori tetap menjadi kompetensi teknis yang harus selalu diasah. Dengan selesainya praktikum ini, praktikan diharapkan mampu mengintegrasikan konsep-konsep ini ke dalam pengembangan aplikasi nyata yang efisien, tangguh, dan skalabel.

BAB VI DAFTAR PUSAKA

- Built In.** *A Guide to Callback Functions in JavaScript*. Diakses dari <https://builtin.com/software-engineering-perspectives/callback-function>
- Codesmith.** *Demystifying Recursion in JavaScript: A Beginner's Guide*. Diakses dari <https://www.codesmith.io/blog/introduction-to-recursion-in-javascript>
- Envato Tuts+.** *Cara menggunakan Map, Filter, dan Reduce di JavaScript*. Diakses dari <https://code.tutsplus.com/id/how-to-use-map-filter-reduce-in-javascript--cms-26209t>
- FreeCodeCamp.** *JavaScript Map, Reduce, and Filter - JS Array Functions Explained with Code Examples*. Diakses dari <https://www.freecodecamp.org/news/javascript-map-reduce-and-filter-explained-with-examples/>
- MDN Web Docs.** (2026, April 15). *Arrow function expressions - JavaScript*. Diakses dari https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions
- MDN Web Docs.** *Functions - JavaScript*. Diakses dari <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions>
- Shift Asia.** *Understanding Immutability: Importance, Use Cases, and Implementation in JavaScript*. Diakses dari <https://shiftasia.com/community/understanding-immutability-importance-use-cases-and-implementation-in-javascript/>